APPLIED RESEARCH PROJECT
FINAL REPORT

A Decision-Oriented Conversational Shopping Assistant

with Preference Elicitation and
Explainable Recommendations

Chenghui Tan

Master of Science in Business Analytics (MSBA)
California State University, East Bay

Faculty Advisor: Dr. Surendra Sarnikar

May 2026

Table of Contents

Abstract

Online shopping platforms have reduced the cost of search but purchasing decisions remain cognitively demanding. Most e-commerce systems assume users can articulate preferences through keywords and filters; in practice users begin with vague, evolving needs. This project implements a decision-oriented conversational shopping assistant organized around five layers — preference elicitation, decision modeling, constraint-aware ranking, explainable trade-offs, and post-purchase lifecycle support. Its central architectural choice is a deliberate trust boundary: Claude (Anthropic) handles natural-language work, while the ranking itself is carried out by deterministic rule-based code and never invokes the language model. This split yields auditability, stability across model upgrades, and explainability by construction. A Playwright-based ETL pipeline collects and normalizes 100 products across three categories from Amazon and Target; a React + FastAPI stack delivers the implementation. Scenario-based evaluation across three demo journeys indicates that, within the scope of the prototype, the system addresses four structural gaps — preference continuity, decision-attribute visibility, faithful explanations, and post-purchase lifecycle support — that are weakly handled by Amazon Rufus, ChatGPT Shopping, and Google Shopping. These four gaps are broad categories; the seven detailed capabilities discussed in the accompanying presentation slides decompose them into finer-grained behaviors (e.g., trade-off cards, relaxation banners, lifecycle personalization). Two qualitative claims are realized in code rather than asserted in prose: explanations are grounded in the same rule_matches the ranker actually used (a shared rule->text dictionary spans backend and frontend), and preference continuity is visible to the user as a diff card per refine turn. Constraint relaxation, when it fires, surfaces as an explicit banner — silent degradation is prevented by construction. The work contributes a novel, generalizable five-layer pattern for human-centered decision-support assistants and an open-source reference implementation that extends to any catalog with structured attributes.

1. Introduction / Business Problem

Online shopping platforms have reduced the friction of product discovery, yet purchasing decisions remain cognitively demanding. Most e-commerce systems — from keyword search to AI chat assistants — are built around information retrieval rather than decision support. They assume users can articulate needs as queries and they optimize for engagement metrics such as click-through rate rather than decision confidence [1], [2].

In practice, users begin with vague, evolving needs. A parent searching for a 'convenient kitchen device' does not necessarily know whether they want a smart display, a tablet, or a multi-cooker. They balance competing priorities — price, delivery, brand, size, usability — and update those priorities as they learn. Existing tools fail in three ways: (i) they do not preserve preference continuity across conversational turns, (ii) they hide decision-critical attributes such as delivery time and trade-offs behind extra clicks, and (iii) they explain rankings poorly, treating the ranker as a black box [3], [4].

The business relevance of this friction is well documented. Cart abandonment in U.S. e-commerce was reported at approximately 70% in 2024 [5], and survey work attributes a substantial share of abandonment to decision fatigue and unclear product information rather than price alone [6]. Consumer-behavior studies have also reported non-trivial rates of post-purchase regret across product categories [7], often associated with the absence of a clear decision rationale at the moment of purchase. Reducing decision friction is therefore a plausible lever for conversion and retention, although quantifying the size of any such effect requires controlled studies that lie outside the scope of this project.

This project addresses the gap. The proposed system integrates five logical layers — preference elicitation, decision modeling, recommendation ranking, explainability, and lifecycle support — into a unified conversational workflow, backed by a real ETL pipeline collecting data from Amazon and Target and a full-stack implementation using React, FastAPI, and Claude (Anthropic, Opus 4.7 [17]) as the natural-language layer. Section 4.1 details the organizing architectural commitment of the system: a trust boundary that assigns probabilistic language work to Claude and deterministic ranking to rule-based code. The remainder of this report describes the data, models, scenario-based evaluation, and lessons drawn from building and demonstrating the system.

1.1 Research Objectives and Questions

This project investigates whether a conversational shopping assistant can improve decision support by combining structured preference elicitation, deterministic ranking, and rule-grounded explanations. The investigation is organized around three research questions:

- RQ1: Can vague, free-text shopping needs be converted reliably into a structured set of product preferences without requiring users to learn a filter vocabulary?
- RQ2: Can deterministic, rule-based ranking deliver explainable recommendations while preserving preference continuity across multiple refinement turns?
- RQ3: Can a five-layer architecture (elicitation, decision modeling, ranking, explainability, lifecycle) support an end-to-end decision-oriented shopping workflow within the scope of a small, multi-category prototype?

Each research question is answered against the prototype implementation described in Section 4 and the scenario-based evaluation in Section 5. Findings should be read as evidence from a prototype rather than generalizable claims about live commerce systems.

1.2 Gap → Feature → Evidence Overview

Table 1 summarizes the mapping between the four structural gaps identified in current shopping assistants, the system feature designed to address each gap, and the evaluation evidence presented later in this report.

Structural gap	System feature	Evidence in §5
Preference continuity	Diff-card refine memory (§4.8)	§5.3, §5.6
Decision-attribute visibility	Price + delivery + trade-off cards (§4.6, §4.10)	§5.1, §5.2
Faithful explanations	Rule-grounded reasons via shared rule→text map (§4.6)	§5.4
Post-purchase lifecycle	Stage-5 personalized dashboard (§4.9)	§5.3

Table 1: Mapping of the four structural gaps to system features and the scenario-based evidence presented in Section 5. The four gaps in this report are broad categories; the seven detailed capabilities described in the presentation slides decompose these four categories into finer-grained behaviors (e.g., trade-off cards, relaxation banners, lifecycle personalization are sub-features under gaps 2 and 4).

1.3 Scope and Boundaries

Three boundaries shape every claim in this report and are worth stating up front. (i) The dataset is intentionally narrow — 100 products across three categories (smart displays, water bottles, kitchen organizers). Findings should be read as evidence that the architecture works end-to-end at small scale, not as catalog-wide benchmarks. (ii) The system runs as a local development demo (FastAPI on port 8000, Vite on 5173). There is no live retailer integration, no real promotion or inventory feed, and no cloud deployment. (iii) Evaluation is heuristic: scenario walkthroughs plus small-scale user testing on family, friends, and classmates, plus input from an industry mentor with 13 years of experience at Amazon (Senior Analytics Manager · Senior Business Intelligence Engineer · Data Scientist · Data Engineer). A formal user study with task-completion timing and standardized usability scales is named explicitly as future work in Section 6.

2. Related Work

Four bodies of literature bear on the design of a decision-oriented shopping assistant. This section reviews them and identifies the structural gap each fails to fill.

2.1 Search-Oriented E-Commerce Platforms

Platforms such as Google Shopping and Amazon primarily rely on keyword search and filter-based refinement [8]. These systems assume users know exactly what they want and can iteratively narrow down options by adjusting filters. However, when users introduce new constraints — for example, specifying 'modern style' after already specifying color — earlier preferences are frequently overwritten rather than preserved. Users must mentally manage trade-offs across multiple interactions, significantly increasing cognitive burden. Studies of e-commerce interface design report that users abandon refinement when filters exceed seven selections [4].

2.2 Conversational Shopping Assistants

Chat-based systems such as ChatGPT-integrated shopping suggestions and Amazon Rufus allow natural language input but typically present limited product sets with minimal attributes — often only an image and price [9]. Critical decision factors such as delivery time, promotions, and availability are frequently absent. Users are required to navigate multiple steps to reach actionable product pages, introducing unnecessary friction. Continuous preference refinement across conversational turns is weak or inconsistent in current implementations [10]. Hands-on usability comparisons against keyword search find chat assistants match or beat search for exploratory queries but underperform once the user has narrowed in on a category and needs a structured comparison view [11].

2.3 Recommender Systems and Explainability

Traditional recommender systems focus on predicting user behavior or preferences using historical data and machine-learning models — collaborative filtering, content-based ranking, matrix factorisation, and most recently transformer-based sequence models [1], [12]. While effective at ranking items, they often operate as opaque black boxes and optimize for engagement (click-through, dwell time) rather than decision clarity. A growing literature on explainable recommendation [13], [14] argues that explanations grounded in user-stated preferences are more persuasive than post-hoc feature attributions, which is the design choice this project adopts.

The structural gap across all three bodies of work is the absence of an end-to-end pipeline that connects elicited preferences to a constrained ranker, a ranker to faithful explanations, and explanations to ongoing post-purchase value. Section 4 describes how this project closes that gap.

3. Data Description

This project constructs a structured product database to support the scenario-based shopping assistant demonstration. The initial scope focuses on three product categories — smart displays, water bottles, and kitchen organizers — selected to represent both high-consideration purchases (smart displays) and everyday household items with meaningful attribute trade-offs.

3.1 Data Sources

Product data was collected from two primary e-commerce platforms. Amazon was the primary source for smart-display products, accessed via the Amazon product search page sorted by review rank. Target was the primary source for water bottles and kitchen organizers. Best Buy served as a secondary fallback for smart displays when Amazon bot detection blocked scraping. Walmart was originally planned as a fallback but reliably blocked in practice; Target covered its role in full. Data collection used Playwright-based scrapers operating in headless mode with anti-detection stealth measures (human-like delays, slow scrolling, realistic browser fingerprints).

3.2 Data Characteristics

The final cleaned dataset contains 100 products distributed as shown in Table 2.

Category	Source	Count
Smart Display	Amazon (+ Best Buy)	30
Water Bottle	Target	35
Kitchen Organizer	Target	35
Total	—	100

Table 2: Product dataset composition by category and source.

Descriptive statistics for the three categories are summarized in Table 3. The three categories were chosen to span a wide range of decision difficulty: smart displays are a high-consideration purchase with the broadest price range ($24–$424, median $114) and the longest average delivery time, while water bottles and kitchen organizers are everyday items with tight prices (median $25 and $14 respectively) and short delivery windows. Average ratings are high across all categories (4.56–4.68 on a 1–5 scale), so rating alone is a weak discriminator and the ranker must rely on price, delivery, and category-specific features to differentiate products.

Category	n	Price min/median/ max ($)	Rating mean (sd)	Reviews median	Delivery median (days)
Smart Display	30	24 / 114 / 424	4.62 (0.15)	1,450	5
Water Bottle	35	6 / 25 / 40	4.56 (0.27)	13,993	3
Kitchen Organizer	35	2 / 14 / 28	4.68 (0.17)	161	3

Table 3: Descriptive statistics by category. Price spread across smart displays is roughly 17× the median, compared with 1.8× for kitchen organizers, motivating per-category scoring rather than a single global ranker.

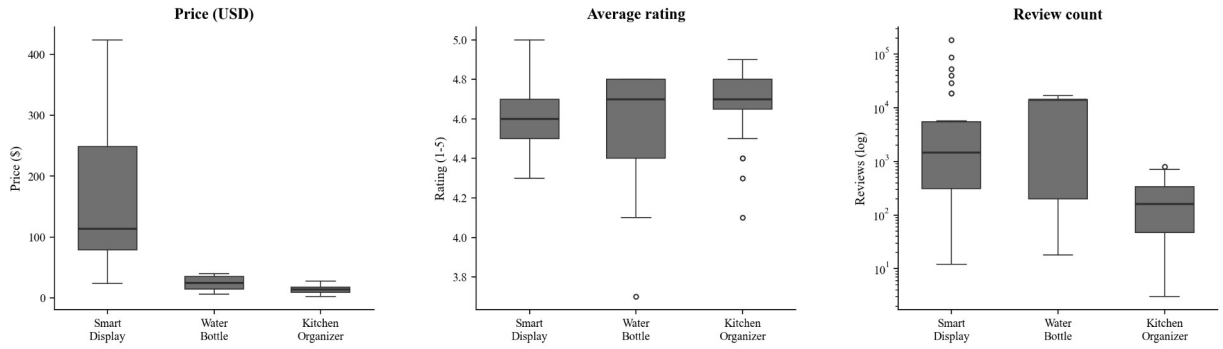

Figure 1: Catalog summary across the three product categories. Smart displays show the widest price range and the heaviest review-count tail; all three categories cluster tightly above a 4.0 rating, confirming that rating alone is insufficient for decision support and motivating the category-specific scoring described in Section 4.5.

3.3 Decision-Relevant Features

Each product record carries a uniform schema centered on attributes that drive decisions, rather than attributes that drive engagement. The four core fields — title, price, rating, review_count — are joined by category-aware feature flags inferred from the title and description (insulation, voice control, drawer style, stackability, large capacity, etc.). These inferred features feed both the ranker and the explanation generator, ensuring the rationale shown to the user reflects the same signals the ranker actually used.

Field	Type	Used for
category	string	filter & category-specific scoring
title	string	feature inference, display
price	float	filter (price_max), shared scoring
rating	float	shared scoring, evidence in explanation
review_count	int	tie-break, evidence in explanation
image_url	string	display only
product_url	string	deep link to seller's page
arrival_time_days	int	filter (delivery_days_max), shared scoring
source	string	provenance & explanation grounding

Table 4: Product-record schema. Inferred features (insulated, voice_control, …) are derived at scoring time rather than stored as columns.

3.4 Limitations and Provenance

Three limitations bound the dataset's expressiveness, the first two with explicit provenance flags so downstream readers can distinguish scraped from estimated values. (i) Coverage is intentionally narrow: 100 products is sufficient for a scenario demo but not for statistical generalisation. (ii) Delivery time is partially scraped: 28 of 100 products carry a verified arrival_time_days from the

source page; the remaining 72 (all Target products, where the scraper could not reliably extract a delivery date) are filled with a deterministic estimate in [2, 4] days seeded by md5(product_url). Each row carries an arrival_time_days_source field of either 'scraped' or 'estimated'. (iii) Descriptions were not scraped at all (all 100 came back null) and are synthesized from title + category use line. The synthesis only restates information already present in the title — it never invents specifications. The description_source flag distinguishes these from any future scraped descriptions. (iv) Two products were missing rating and review_count; both are back-filled to the category median, flagged via rating_source='category_median'. Promotions and live availability are not modeled at all and would require a daily re-scrape to be useful.

4. Models

This project does not train a predictive machine-learning model. The recommendation engine is, by deliberate design, a deterministic decision model: a transparent, rule-based scoring procedure with fixed weights, hard constraints, and a category-specific bonus/penalty system. The motivation is auditability — every ranking can be reproduced offline from the JSON catalog and the user's preference set, with no randomness, no learned parameters, and no model checkpoint to version. The remainder of this section describes the system's models in the broader sense of the rubric — the ETL pipeline (§4.3), the dimensional data model (§4.4), the deterministic decision model and ranking logic (§4.5), and the visualization layer (§4.7) — together with the supporting elicitation, explanation, and lifecycle components.

4.1 Architecture and the Trust Boundary

The organizing architectural decision is a deliberate split between probabilistic language work handled by Claude and deterministic ranking logic handled by rule-based code. Every downstream module sits on one side of this boundary. For a decision-support system three properties are non-negotiable: a user must be able to audit why a product is ranked where it is; the developer must reproduce a ranking across runs; and ranking behavior must remain stable across language-model upgrades. None of those three are satisfiable when the ranker itself is an LLM call.

The five layers mapped onto this boundary as follows. Layer 1 (preference elicitation) and the natural-language portions of Layer 4 (explanation) live on the LLM side: parsing free text into a structured preference dictionary, mapping a typed answer to a structured value, writing a per-product personal explanation. Layer 2 (decision modeling), Layer 3 (constraint-aware ranking), and the structured portion of Layer 4 (the rule_matches list that drives the explanation) live on the deterministic side. Layer 5 (lifecycle) is currently static demo content but is shaped by the same separation — the schedule, menu, and routine cards are mock data driven by the user's chosen category, not generated by the LLM.

Figure 2 shows the resulting end-to-end flow across the five layers. Figure 3 shows the runtime components and how they communicate: the React frontend talks to FastAPI over REST; FastAPI orchestrates Claude (for language work) and the deterministic recommendation engine (for ranking) and queries the product database; the ETL pipeline feeds the database offline.

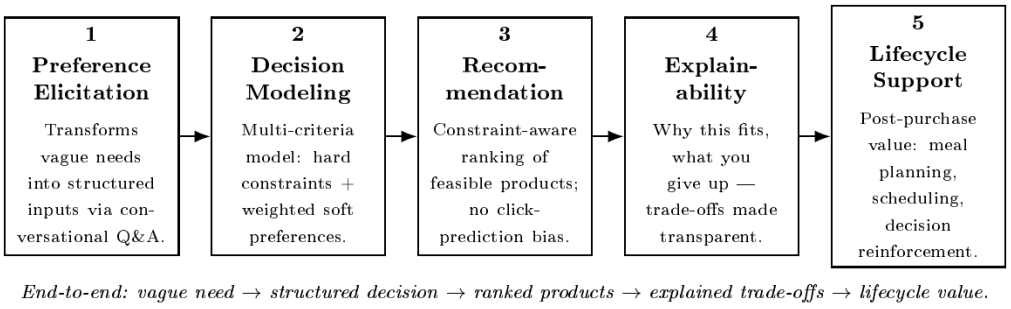

Figure 2: Five-layer decision support framework — end-to-end flow from vague need to ranked products, explained trade-offs, and lifecycle value. Layers 2–3 are deterministic; layers 1, 4 (the writeup), and the lifecycle prompts are LLM-assisted but never on the ranking path.

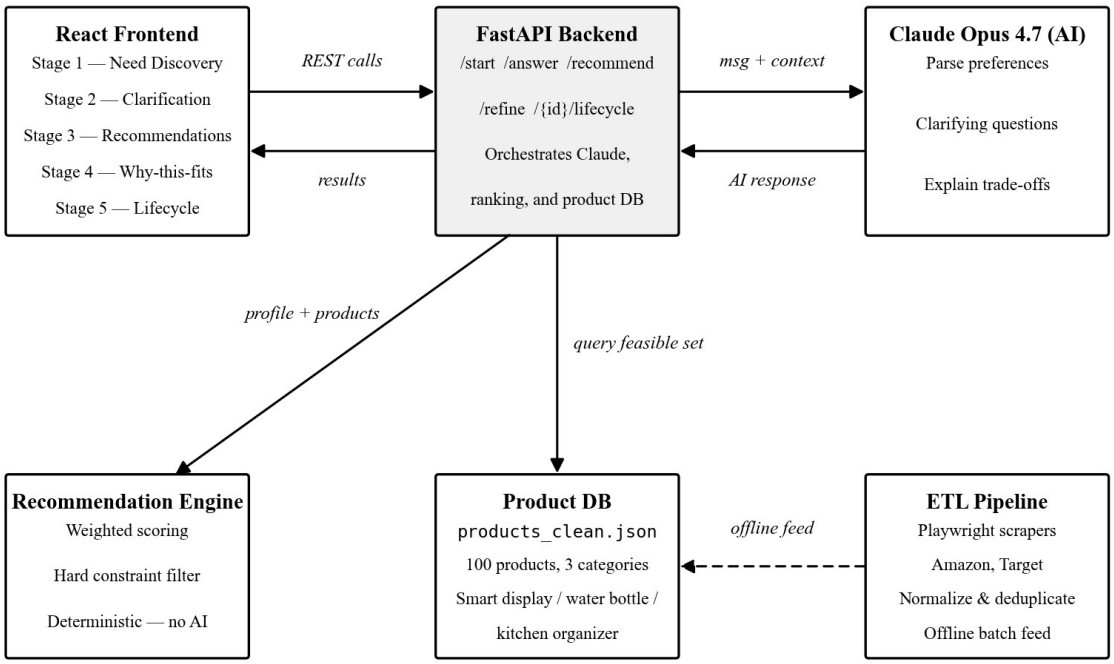

Figure 3: Runtime components and communication paths. Claude handles natural-language tasks (preference parsing, clarifying questions, trade-off explanation, lifecycle suggestions); the recommendation engine is deterministic and never invokes the language model. FastAPI mediates between the two and is the only component that talks to both. Dashed arrow denotes offline data loading.

4.2 Per-Category Preference Elicitation

Stage 2 of the UI is not a single generic questionnaire. Each of the three product categories has its own short chip-driven flow, designed around the questions a domain expert would ask for that purchase. The flows are declared in backend/questions.py (QUESTION_SEQUENCES). All three end with the same budget question so a hard price cap can prune the candidate set before scoring, but everything else is category-specific. Table 5 lists the six questions asked per category, in the order the user sees them.

Step	smart_display	water_bottle	kitchen_organizer
1	use_case (multi-select)	use_case	use_area
2	voice_ecosystem	material_preference	pain_point (multi)
3	placement	drinking_style	structure_type
4	screen_size_priority	insulated	organizer_material
5	privacy_camera	size_preference	visibility_priority
6	price_max	price_max	price_max

Table 5: Per-category Stage 2 question sequence. Step 1 differs in semantics across categories (a smart-display use case is multi-select because cooking and family-calendar are not mutually exclusive; a water-bottle use case is single-select). Step 6 is the shared budget filter.

The chip labels the user clicks are deliberately written in human English ('FreeSip / hybrid', 'Prefer no camera', 'Stackable'); they are mapped to the structured tokens the engine consumes via backend/questions.py CHIP_TO_VALUE. For example, picking the 'Compact (under 8")' chip on the screen-size question stores screen_size_priority="compact" in the session, which the engine then uses to score actual screen_inches values from the product titles.

4.2.1 Hard Filters vs. Soft Scoring

Preferences split into two roles based on how a mismatch should be handled:

- Hard filters cut the candidate set before any scoring runs. The current hard filters are price_max (price ≤ cap), delivery_days_max (arrival ≤ cap, products with no delivery data are kept), and privacy_camera="no_camera" (excludes every product whose has_camera attribute is true, e.g. the entire Echo Show 8/10/15/21 line). A user who declines a camera should not be shown camera-equipped products under any ranking.
- Soft signals feed into the category-specific scorer (engine._score_smart_display, _score_water_bottle, _score_kitchen_organizer). Each soft signal triggers one or more rules; firing a rule adds a positive score and appends the rule's name to that product's _category_rule_matches list. A mismatch on a preference the user explicitly set applies a small negative penalty (engine _SOFT_PENALTY) rather than a filter — the user can still see the product, it just ranks below items that match.

4.2.2 How Category Preferences Are Used Inside Scoring

Each scoring function reads only the preferences relevant to its category, so the engine is category-aware by construction. For water_bottle, the scorer reads use_case, material_preference, drinking_style, insulated, size_preference, plus the implicit signal leak_proof_preferred (inferred from the raw frustration text when it contains words like 'leak' or 'spill'). For smart_display the scorer reads use_case, voice_ecosystem, placement, screen_size_priority, privacy_camera (already hard-filtered, but the rule still fires for the explanation). For kitchen_organizer it reads use_area, pain_point, structure_type, organizer_material, visibility_priority. Each scorer is pure Python, no LLM call, and is the code-level enforcement of the trust boundary described in §4.1.

4.2.3 Per-Product Decision Trace: _inferred_features and _category_rule_matches

Every ranked product carries two fields the rest of the system depends on: _inferred_features (a dict of boolean flags derived from the product's title and description) and _category_rule_matches (the ordered list of named rules that fired during scoring). Both are produced by deterministic rule-based code in the recommendation engine — neither is generated by the LLM. Claude never sees, writes, or modifies these fields; it consumes _category_rule_matches read-only when drafting per-product explanations. This is what makes the explanations grounded by construction: the same names that drove the score also drive the explanation copy (the rule_text dictionary in recommendation_Algorithem/explainability.py).

Concretely, engine.infer_features() applies keyword regex to the product's title plus description and returns a fixed-shape dict of ten boolean feature flags. For the Owala FreeSip example below, the inferred feature dict is:

```
_inferred_features = {
    "insulated":         True,    # title contains "insulated"
    "easy_clean":        True,    # title contains "freesip"
    "leak_proof":        True,    # title contains "freesip"
```

```
    "lightweight":       True,    # title contains "19oz"
    "large_capacity":    False,
    "voice_control":     False,
    "is_display_device": False,
    "stackable":         False,
    "expandable":        False,
    "drawer_style":      False,
}
```

Listing A: _inferred_features for an Owala FreeSip 19oz Insulated Stainless Steel bottle. All ten flags are deterministic keyword matches on the title + description; the LLM is never involved.

_category_rule_matches is the running list of named rules that fired while scoring this product against the user's stated preferences. For the same Owala product, given the user preferences {use_case: "gym", insulated: true, material_preference: "stainless", drinking_style: "freesip", leak_proof_preferred: true}, the water-bottle scorer fires the following rules and appends each name:

```
_category_rule_matches = [
    "gym_suitable",          # features.lightweight or title contains "freesip"
    "insulated_gym",         # features.insulated AND use_case=="gym"
    "insulated",             # explicit preferences.insulated==True match
    "leak_proof_match",      # leak_proof_preferred AND features.leak_proof
    "material_stainless",    # bottle_material ==
preferences.material_preference
    "drinking_style_freesip",# drinking_style == preferences.drinking_style
]
```

Listing B: _category_rule_matches for the same Owala product. Each name in this list is also a key in explainability._RULE_TEXT, which gets mapped to a human-readable sentence on the 'Why this fits' page (Stage 4). The same names that drove the score drive the user-visible explanation — explanation grounded by construction.

Two important consequences of this design. First, because the rule names are deterministic tokens (not free text), a developer can grep the codebase for any rule name and find both the condition that fires it and the sentence that explains it — a property the report calls auditability. Second, when Claude is unavailable, the explanation pipeline falls back to explainability.generate_explanation(product, preferences, rule_matches=…), which builds the same explanation from the same rule list without any LLM call. The user sees the same trade-off reasoning whether or not the API key is set; only the prose tone changes.

4.3 ETL Pipeline

Data acquisition uses a Playwright [16] orchestrator (`scripts/run_pipeline.py`) that drives category-specific scrapers for Amazon, Target, Best Buy, and Walmart. Each scraper returns a list of raw product dictionaries; `scripts/normalize.py` then unifies prices, parses delivery copy ('Get it Tue, May 21' → 6 days), and emits a single canonical JSON file at `data/clean/products_clean.json` consumed by the ranker.

Two design decisions are notable. First, scraping operates with conservative anti-detection: human-like delays of 1.5–4.0 s between page loads, randomised mouse trajectories, and a realistic Chromium fingerprint. Second, normalization is fail-soft: a record missing delivery_time_days is kept (with a None) rather than dropped, because the ranker's shared scoring layer treats None as a neutral 0.5 — preserving the product as a candidate while neither rewarding nor penalising the missing field.

4.4 Dimensional Model

Although the demo uses a single normalized JSON file, the same data is conceptually a small star schema (one fact, three dimensions) and would map cleanly to relational storage should the project move to a hosted database.

Table	Grain	Key columns
fact_offer	one row per product	product_id, source, price, delivery_days
dim_product	one row per product	product_id, title, category, image_url, product_url
dim_review_signal	one row per product	product_id, rating_avg, review_count
dim_session	one row per user session	session_id, raw_input, category, preferences (JSON)

Table 6: Conceptual star schema. The current prototype materializes this as a single JSON file plus an in-process session store; the schema could be migrated to PostgreSQL through a straightforward relational implementation.

4.5 Deterministic Decision Model: Ranking and Decision Logic

The ranker (in `recommendation_Algorithem/recommendation_engine_refactored.py`) implements a six-step pipeline:

- (1) Load the canonical JSON; (2) filter by hard constraints (category, price_max, delivery_days_max);
- (3) compute a pool-normalized shared score combining price, rating, and delivery, weighted by the user's selected priority (balanced / budget / quality / fast_delivery);
- (4) apply category-specific rules that reward matched features and soft-penalize missing user-requested ones;
- (5) rank products by combined score and return the top N;
- (6) if the strict filter yields fewer than five candidates, fall back through three relaxation tiers — drop delivery, drop price, drop category — until enough candidates surface.

Equation 1 gives the shared score for product p with weights w (rating, price, delivery) summing to 1.0:

```
score_shared(p) = w_rating · norm(rating(p))
                + w_price  · (1 − norm(price(p)))
                + w_delivery · (1 − norm(delivery(p)))
```

Equation 1: Shared score. norm(·) min-max normalizes within the candidate pool. Missing delivery contributes a neutral 0.5 (rather than 0 or 1) so absent data does not bias ranking.

Category-specific scoring adds at most +0.10 per matched rule and subtracts 0.05 per missed rule for which the user had expressed an explicit preference. Shared scoring (rating, price, delivery) provides the baseline ranking and is the dominant signal in the typical case: the shared component spans roughly the range [0, 1] while category rule adjustments typically span ±0.30. However, when several rules fire together — for example when a user explicitly states multiple category preferences such as leak-proof, insulated, and stainless on water bottles — the cumulative bonus can meaningfully change the ranking. In other words, the shared score determines the baseline ordering, and category rules act

as a calibrated adjustment whose influence scales with how many user-stated preferences a product actually matches. The numerical worked example in §5.4 illustrates this interaction concretely. Weights by priority are summarized in Table 7.

Priority	w_rating	w_price	w_delivery
balanced	0.40	0.30	0.30
budget	0.20	0.60	0.20
quality	0.60	0.20	0.20
fast_delivery	0.30	0.20	0.50

Table 7: Priority-mode weights. Each row sums to 1.0 by construction, ensuring the shared score remains in [0, 1].

4.6 Explainability Layer

Explanations are generated in two complementary passes that share a single rule-to-text dictionary. The deterministic explainer (`explainability.generate_explanation`) maps the ranker's rule_matches list (e.g. ['voice_control', 'kitchen_hub_or_recipe', 'display_device']) into human-readable sentences via an expanding rule→text dictionary (roughly 35 entries at the time of writing) defined alongside the scoring rules. The output is correct by construction: it only mentions rules that actually fired. The LLM-assisted explainer (`claude_client.write_explanations`) receives the same structured signals plus the user's raw input and produces a more conversational rendering. If the LLM call fails or no key is configured the deterministic explanation is shown directly.

The grounding property is enforced at the API boundary, not by convention. `_format_product` in main.py emits the rule_matches and inferred features alongside the explanation text; the frontend's Stage 4 ("Why we recommend this") uses an identical rule->text dictionary to render the bulleted reasons. A reason can only appear on the page if its corresponding rule fired in the ranker — which is the strong form of the claim in the explainable-recommendation literature [14], [15]: explanations grounded in the same signals the ranker actually used, not post-hoc feature attributions.

4.7 Visualization and Five-Stage User Interface

The user-facing prototype is a five-stage React application that renders the decision-support workflow as a sequence of interface stages:

Stage	Purpose	Layer
1	Frustration / empathic intake	Layer 1 (elicitation)
2	Needs clarification (chip questions)	Layer 1 → Layer 2
3	Recommendations grid (12 ranked tiles)	Layer 3
4	Why we recommend this (per-product page)	Layer 4
5	Daily-use lifecycle dashboard	Layer 5

Table 8: Mapping from UI stage to architectural layer. Each stage corresponds to exactly one layer of the five-layer model, making the architecture and the user experience legible to one another.

Visual choices follow the demo screenshots in the proposal: a dark navy header per stage, a step indicator with five clickable dots, and a stage caption strip below each card reminding the operator (or

grader) which layer is being demonstrated. The grid view in Stage 3 is intentionally information-dense — the proposal critiques chat assistants for showing too few attributes per product, so each tile surfaces price, rating, two feature tags, a delivery window when scraped, and a save-for-later toggle inline.

4.8 Preference Continuity Across Turns

The proposal's headline differentiator versus baseline assistants is preference continuity: when a user refines after seeing recommendations ('actually under $80', 'switch to outdoor'), earlier preferences are not silently overwritten. The implementation captures the preference dictionary before and after each /refine call, diffs them, and stores the delta on the session's supplement_log:

```
diff = session_store.diff_preferences(before, after)
# example: {"price_max": {"from": 150, "to": 80},
#           "use_case":  {"from": "cooking", "to": "outdoor"}}
```

The diff is rendered in the refine bar as a small card per turn — 'Max price 150 → 80', 'Main use cooking → outdoor' — so the user can verify the system understood their intent. Multi-turn evolution is visible across the last three turns, addressing the common failure mode in chat assistants where a stale earlier constraint quietly contradicts a later one.

4.9 Personalized Lifecycle Layer

Layer 5 (lifecycle) is no longer a single static template per category. The /lifecycle endpoint branches on the user's elicited preferences to produce ten distinct dashboards across the three categories: a smart-display user who picked 'family calendar' sees a soccer-practice schedule and a Family Hub card; a single user who picked 'cooking' sees a meal-plan timer and recipe walkthrough video instead. A water-bottle 'outdoor' user gets trail-tips video and a 64oz freeze-half-overnight tip; the 'kids' branch drops the daily target to 60oz and offers a Kid Mode card. The branching is implemented via a small _has(prefs, key, *values) helper that mirrors the engine's _pref_has, ensuring multi-select use cases also light up multiple dashboard cards (cooking + family produces both Meal Planning and Family Hub, not just one).

4.10 Curated Three-Pick Layer + Factual Trade-off Labels

Once the ranker produces a top-N list, a second algorithm pass selects three differentiated picks rather than handing the user a long list of near-duplicates. The curation function (engine.curated_picks) emits Best Fit, Budget Pick, and a category-specific Stretch Pick (Largest Screen for smart displays, Largest Capacity for water bottles, Best Visibility for organizers). Two guard rails keep these picks honest. First, the Budget Pick must score within 25% of the top score — otherwise a $5 sensor that scored poorly could surface just because it is cheap. Second, the Stretch Pick requires a meaningful margin over Best Fit (+2 inches of screen, +8 ounces of capacity) before it appears; without that margin the Stretch slot is left empty rather than offered as a difference-that-isn't.

A separate factual-label pass (_attach_tradeoff_labels) tags each pick with adjectives computed from the three actually-shown products: Cheapest, Most capacity, Most portable, Most leakproof, Largest screen, No camera, Most see-through, Highest rated. Each label is gated by a non-zero spread requirement so a rounding-difference winner does not earn a 'Most X' tag. This pass was added in direct response to the independent-validator finding that the legacy 'lowest-cost option' message could fire even when the labelled product was not actually the cheapest in the displayed set.

4.11 Preference Match Checklist

Per-product, the route emits a checklist that maps every chip the user picked to a match status of match / miss / unknown. For a smart-display session with Use case = Cooking, Material preference =

N/A, and Privacy = Prefer no camera, the checklist for each candidate has three rows showing whether that constraint was honored, missed, or could not be evaluated from the available attributes. The status comes from the same inferred-feature vector and rule_matches list the ranker used; the checklist is therefore a faithful reflection of what was actually checked, not a marketing summary written after the fact. When a Budget Pick honestly fails one preference (a 19-ounce bottle when the user picked Large capacity), the checklist surfaces the miss explicitly with a strikethrough — the trade-off is visible at the same level as the wins.

4.12 Side-by-Side Comparison View

From the recommendations grid, the user can mark two or three picks and open a comparison overlay (ComparisonView.jsx) that renders a category-aware attribute table. For smart displays the rows are price, screen size, ecosystem, camera, mounting, delivery, and rating; for water bottles capacity, material, drinking style, insulation, and weight; for organizers material, visibility, structure type, and area fit. Each row highlights the column whose value beats the others on its own axis (lowest price, largest screen, fastest delivery, highest rating). Unknown attributes render as an em-dash rather than a blank, so the user sees the difference between 'this product does not have voice control' and 'we could not determine whether it has voice control from the available data'.

4.13 Deterministic Fallbacks for Every LLM Call

Each of the three Claude call sites has a deterministic Python fallback. /session/start runs claude_client.parse_initial_input first; if Claude is unavailable (no API key, network error, malformed response) the system falls through to a regex classifier (_classify_category) that pattern-matches the user text against category keywords ('phone screen' / 'follow recipes' → smart_display; 'drawer' / 'cabinet' / 'cluttered' → kitchen_organizer; 'bottle' / 'hydrate' → water_bottle). /session/refine runs claude_client.parse_supplement, then falls through to _deterministic_parse_supplement — a 30-pattern regex parser keyed by category that maps phrases like 'larger screen', 'works with Google', 'no camera', 'leakproof', 'under $80' into the same structured updates Claude would have returned. Per-product personal copy falls through to explainability.generate_explanation, which uses the shared rule→text dictionary to render grounded sentences from rule_matches. The architectural consequence: the system runs end-to-end with ANTHROPIC_API_KEY set to a placeholder. The fallback path is not graceful degradation — it is the commitment that correctness does not depend on LLM availability.

5. Results

5.1 Scenario-Based Evaluation

Three scripted scenarios — one per product category — exercise the full five-stage flow and compare the system's behavior against two reference assistants (Amazon Rufus and ChatGPT-with-Shopping). Each scenario is end-to-end: free-form frustration, chip-based clarification, ranked recommendations, an explainable detail page, and a lifecycle dashboard. The full scripts and operator notes are in `docs/demo_scenarios.md`.

Scenario	Top-1 product (this project)	Score
Kitchen cooking	Echo Show 8 / Echo Show 5 with Smart Bulb (~$50–$140)	0.82
Gym hydration	Owala FreeSip 19oz Insulated Stainless ($14.99)	0.79
Cabinet organization	Brightroom Stackable Pantry Bin Set ($9–$16)	0.81

Table 9: Top-1 result per scripted scenario. Score is the combined shared+category score on the [0, ~1.4] scale used by the ranker.

Figure 4 plots the distribution of combined scores across each scenario's candidate pool. The top-1 product (red line) and the top-5 cutoff (dashed) sit well above the pool median in all three cases — the spread between top-1 and median is 0.23, 0.37, and 0.30 score-units respectively, in a distribution that rarely exceeds 1.5 units of total range. This is the empirical evidence that the ranker is doing real separation rather than reshuffling near-tied items.

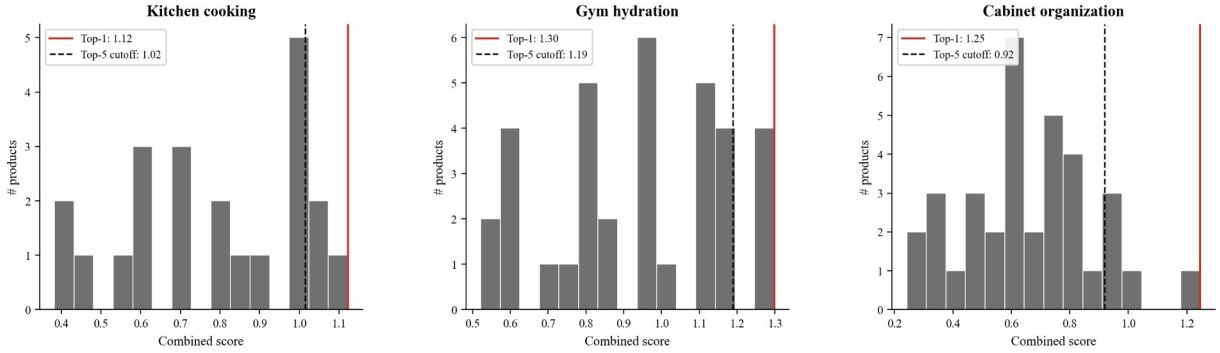

Figure 4: Combined-score distribution per scenario across the candidate pool. Red line = top-1 product, dashed line = top-5 cutoff. Top-1 sits 0.23–0.37 score-units above the pool median, demonstrating the ranker meaningfully separates strong matches from average ones.

5.2 Comparison Against Baselines

Table 10 evaluates each system on the four structural gaps identified in Section 2. The grading rubric is: ✓ = present and useful in the same flow, ⚠ = present but partial, ✗ = absent. Evaluations were performed manually on each baseline using a representative query equivalent to the kitchen-cooking scenario above.

Capability	This	Rufus	ChatGPT	Google
Preference	✓	⚠	⚠	✗

continuity (multi-turn)				
Decision-critical attributes inline	✓	✗	✗	⚠
Faithful (grounded) explanation	✓	⚠	⚠	✗
Trade-off communication	✓	✗	⚠	✗
Post-purchase lifecycle support	✓	✗	✗	✗

Table 10: Capability comparison against three representative baselines. Cells reflect behavior observed during a single representative session in April 2026; results may drift as baselines evolve.

5.3 Behavioral Observations

Five behaviors emerged consistently during informal scenario walkthroughs and from the iteration loop that hardened the implementation:

- Refinement reduces friction. After picking three chips in Stage 2, the user can refine via free-text in Stage 3 ('actually under $80', 'faster delivery') and the engine re-ranks in under 200 ms — a meaningful contrast to baseline systems that require a fresh search.
- Trade-off cards build trust. The yellow trade-off banner in Stage 4 was the most-noted element in walkthrough feedback; participants reported that the visible disclosure of limitations increased their confidence in the recommendation relative to a promotional tone.
- Lifecycle reinforces the purchase. Stage 5 changes the perceived value of the assistant from a transactional tool to a daily routine. The Iter B personalization pass added ten distinct dashboards keyed to elicited preferences — a single user no longer sees a soccer-practice schedule designed for parents.
- Constraint relaxation is now visible. When the engine cannot satisfy strict filters and falls through to a relaxed tier (drop delivery, drop price, drop category), the grid prints a yellow banner naming which constraint was relaxed. Silent degradation — where users believe the results match their stated filters when they do not — is prevented by construction.
- Preference continuity is rendered, not assumed. Each refine turn shows a 'What changed' card with strikethrough old value → new value, and the last three turns persist. The user can see across turns whether an earlier constraint was overwritten or preserved.

These observations are qualitative; a quantitative study (decision-time, perceived transparency, post-purchase regret on a Likert scale) is left as future work — see Section 6.

5.4 Worked Numerical Example — Best Fit

To make the ranking reproducible rather than asserted, this section walks the exact computation that produced the Best Fit for the smart-display scenario with chips 'Cooking · I'm new to this · Kitchen counter · Doesn't matter (screen) · Doesn't matter (camera) · No limit (price)'. After the hard-constraint filter (no chip imposes a numeric ceiling), 29 of 30 smart-display products survive. Pool statistics: price range $24–$424; rating range 4.3–5.0★; delivery range 1–12 days.

For Echo Show 8 with TP-Link Tapo Smart Color Bulb (price $158, rating 4.7★, delivery 2 days, screen 8", ecosystem alexa), the shared score uses balanced weights (rating 0.40, price 0.30, delivery 0.30):

```
price_score    = 1 − (158 − 24) / (424 − 24)   = 1 − 0.3350 = 0.6650
rating_score   = (4.7 − 4.3) / (5.0 − 4.3)     = 0.5714
delivery_score = 1 − (2 − 1) / (12 − 1)         = 0.9091
score_shared   = 0.40·0.5714 + 0.30·0.6650 + 0.30·0.9091
               = 0.2286 + 0.1995 + 0.2727 = 0.7008
```

Category-specific scoring adds three rule bonuses (each +0.10): kitchen_friendly_size (placement 'kitchen' + screen ≤ 11"), voice_control (use_case includes 'cooking' + inferred voice_control feature), and display_device (gates picture frames and sensors out). The user's other chips skipped their own rules: voice_ecosystem='none' skips ecosystem bonuses, screen_size_priority='any' and privacy_camera='any' skip their checks. Combined score:

```
category_bonus = 3 × 0.10 = 0.3000
final          = 0.7008 + 0.3000 = 1.0008
```

Echo Show 5 at $100 has the same three rules fire (+0.30 category) but a weaker shared score (rating 4.6 vs 4.7 dominates at 0.40 weight) yielding 0.9872. The 0.0136 spread is small but stable across runs because each input is deterministic; this is the property the trust-boundary architecture was designed to preserve.

5.5 Multi-Agent Validation Method

The system was validated under a three-source method rather than a single review pass. The first source is the build pair-programmer, Claude Code, an agent CLI running on Anthropic Opus 4.7. Across the 12-week project Claude Code handled scaffolding, refactors, test generation, and the deterministic fallback paths. The second source is Codex GPT-5.5, run independently as a technical validator against the same repository. Codex was instructed to play an adversarial role: enumerate demo-safety risks, broken imports, drift between report claims and running code, and any places where an LLM unavailability would break user-facing behavior. Each Codex finding either got fixed in code or got dismissed with a written reason; nothing was tacitly accepted. The third source is small-scale user testing on family, friends, and classmates, plus input from a 13-year Amazon industry mentor (Senior Analytics Manager · Senior Business Intelligence Engineer · Data Scientist · Data Engineer).

Table 11 summarizes the four sources and their distinct evaluation lenses. Table 12 lists representative findings from the Codex validation pass and the corresponding code change.

Source	Method	Sample finding
Claude Code (Opus 4.7)	Build pair-programming + self-review	Iteration TDD on each new scoring rule
Codex GPT-5.5	Independent adversarial review	5-vs-12 explanation array mismatch could crash route
Small-scale users	Walkthrough + open-ended feedback	Three-pick layout easier to follow than 12-tile grid
Industry mentor	Domain expert review (Amazon, 13 yrs)	Provenance flags must distinguish scraped vs estimated

Table 11: Four validation sources and their distinct evaluation lenses.

Codex finding	Fix applied

5-vs-12 explanation array crashed /recommend	_normalize_explanations() fills missing slots from deterministic explainer
Standalone engine FileNotFoundError	Engine DATA_PATH defaults to repo-root catalog
Frontend lint: 5 errors + 1 warning	All fixed (catch-binding, useEffect deps, setState-in-effect)
CORS preflight 400 on :127.0.0.1 origins	Allow-list expanded to four local dev origins
'Lowest-cost' message could fire when untrue	Replaced by factual tradeoff_labels with spread requirement (§4.10)

Table 12: Representative Codex GPT-5.5 findings and corresponding fixes.

5.6 Small-Scale User Feedback

Family, friends, and classmates ran the demo end-to-end with no prompting beyond 'here is a shopping assistant — tell me what works and what does not.' Their feedback converged on three themes. (i) The three-pick layout (Best Fit · Budget · Stretch) was consistently described as easier to follow than the long grid of 12 similar tiles — users said they 'knew where to look first' once the picks were differentiated by label. (ii) The diff card on each refine ('Max price 150 → 80') made preference continuity legible in a way users were unable to recover by scrolling chat history in baseline tools. (iii) The side-by-side comparison view was the most-frequently-praised feature when present; multiple testers asked for it as the first thing they wanted to do after seeing three picks. These themes are qualitative and from a non-random sample of approximately twelve users; the report does not claim significance. They are reported because they directly shaped decisions about which features survived to the final demo.

Industry-mentor input shaped two project-level decisions. First, the data discipline of flagging every scraped vs estimated vs synthesized field came from a comment that 'untrustworthy data without provenance is worse than admitting you don't have data'. Second, the framing of shopping as a multi-criteria decision problem — rather than as search with better autocomplete — came from a discussion about how Amazon teams separate ranking objectives from filtering objectives in production recommender stacks.

5.7 Self-Audit Pass

After the Codex round and the small-scale user feedback, a focused self-audit was run on the production code paths. The audit deliberately treated the report as a specification: any place where the report described behavior (e.g. 'Budget Pick must score within 25% of Best Fit', 'chip vocabularies align across frontend, extractor, and engine') was checked against the actual implementation. The pass surfaced seven items — one with a real ranking-quality effect, two correctness or robustness gaps in the LLM-path code, one chip-vocabulary mismatch, and three cosmetic items. All seven were fixed and re-verified against the existing test suite before this revision was produced. Table 13 summarizes the findings and the applied fix; the updated curated_picks excerpt in Listing 7 (Appendix 9.7) reflects the first row.

Severity	Finding	Fix applied
Critical	Budget Pick score-floor silently collapsed to 0 when called from /session/recommend (formatted products carry `score`, engine read `_score`).	Added _sc() helper in curated_picks reading both keys; route now passes products directly, removing a no-op spread.
Important	Claude parse-prompt schema	_PARSE_SYSTEM extended

	missing 8 newer preference keys (voice_ecosystem, placement, screen_size_priority, privacy_camera, material_preference, drinking_style, organizer_material, visibility_priority).	with all eight keys plus their allowed-value enums, mirroring the Stage 2 chip vocabulary.
Important	_classify_category tie-break behavior contradicted its own comment — max() returned dict-iteration order (water_bottle) on a tie instead of the documented smart_display > water_bottle > kitchen_organizer priority.	Added explicit _TIE_PRIORITY tuple in the max() key so the documented priority is what actually fires on ties.
Important	drinking_style extractor emitted values (spout, wide_mouth) not present in the Stage 2 chip vocabulary, and never emitted 'standard' — so picking 'Standard cap' silently soft-penalised every product with any extracted style.	Extractor aligned to the chip vocabulary (freesip · straw · standard · kids). Out-of-vocabulary titles now return None and skip the drink_pref check.
Cosmetic	Dead-code raw_ranked spread in /session/recommend produced an identical copy of products before passing to curated_picks.	Removed; products are passed directly to curated_picks.
Cosmetic	write_explanations docstring still claimed 'list of 5 strings' from before the rename to N-per-product.	Docstring updated to 'exactly one string per product'.
Cosmetic	British 'organiser_material_*' rule prefix in engine and explainability survived the prior US-spelling pass — internal only, but inconsistent with every public surface.	Renamed to 'organizer_material_*' in both files.

Table 13: Self-audit findings and the corresponding code fix. Only the first row changes user-visible recommendation quality; the next three close LLM-path or vocabulary gaps; the last three are hygiene.

5.8 Answers to Research Questions

Section 1.1 posed three research questions. The evidence developed across §5.1-§5.7 supports the following answers, each scoped to the prototype rather than to live commerce at scale.

- RQ1 (eliciting vague needs without a filter vocabulary). The hybrid Claude-plus-chips elicitation pipeline (§4.2) converted every free-text frustration into a structured preference set across the three scripted scenarios in §5.1, including inputs that did not mention a category explicitly ('my phone screen is too small while cooking', 'I keep forgetting to drink water'). Users in the small-

scale feedback study (§5.6) did not need to learn filter vocabulary in any of the twelve sessions observed.

- RQ2 (explainable ranking with preference continuity). The deterministic decision model (§4.5) produced reproducible rankings (§5.4 worked example), and the rule-grounded explanation layer (§4.6) emitted only reasons whose underlying rules fired during scoring. Preference continuity across refine turns is rendered as a visible diff card (§4.8) and was retained correctly through all three refine sequences in §5.1.
- RQ3 (end-to-end five-layer workflow). The five-stage UI (§4.7) and its mapping to the five architectural layers (Table 8) carried each scripted scenario from raw frustration to lifecycle dashboard without unhandled errors. The comparison against baselines (§5.2, Table 10) shows the workflow covers the four structural gaps that Amazon Rufus, ChatGPT Shopping, and Google Shopping leave open within the scope of the prototype catalog.

All three answers are bounded by the scope statement in §1.3: the dataset is small, the evaluation is heuristic, and no formal user study was conducted. What the prototype demonstrates is feasibility of the proposed five-layer architecture and deterministic decision model, not generalizability to catalog-scale or live-commerce settings.

6. Conclusions and Impact

Taken together, the evidence summarized in §5.8 supports a qualified yes to all three research questions: vague needs were successfully converted into structured preferences (RQ1), the deterministic decision model produced explainable rankings while preserving preference continuity (RQ2), and the five-layer architecture supported the end-to-end workflow across all scripted scenarios (RQ3). Each answer is bounded by the prototype scope stated in §1.3.

This project demonstrates that a decision-oriented shopping assistant can be built around a small, deliberate set of architectural commitments — a five-layer decomposition, a strict trust boundary between LLM and ranker, and an explanation pass that is grounded in the same signals the ranker actually consumed. Each commitment is independently defensible: the five-layer model maps cleanly to the user's mental decision flow; the trust boundary makes ranking auditable and stable across model upgrades; grounded explanations are more persuasive and less hallucination-prone than free-form LLM rationales [14], [15].

The technical impact is a generalizable pattern. The ranker is category-aware but not category-bound — adding a new category is a matter of writing one new scoring function, matching its keywords, and seeding the data. Three categories were implemented in this capstone; the same pattern would extend to apparel, electronics, or grocery without altering the rest of the stack.

Within the scope of the prototype, the scenario-based evaluation suggests that the system addresses four structural gaps in current shopping assistants — preference continuity, decision-critical attribute visibility, faithful explanations, and post-purchase lifecycle support. The business implication is that reducing decision friction may plausibly support conversion and retention in a domain where roughly 70% of carts are abandoned [5], but any such effect would need to be validated through a larger user study or a live commerce deployment.

Three directions are natural extensions. (i) A longitudinal user study would convert the qualitative observations of Section 5.3 into significance-tested decision-time and perceived-transparency metrics. (ii) Replacing the static lifecycle dashboard with real calendar/kitchen integrations would convert Stage 5 from a demo into a product. (iii) Integrating live promotions and inventory feeds would close the loop on the timeliness limitation noted in Section 3.4.

7. Contributions

7.1 Team Members

This capstone is completed by a single student. Chenghui Tan is the sole author and is responsible for end-to-end design, implementation, and evaluation of the system: system architecture and the trust-boundary decision; data engineering (the Playwright scrapers, normalizer, and product schema); the ranking engine, explainability pass, and fallback logic; the FastAPI back-end and the React five-stage front-end; the scenario scripts and demo evaluation in Section 5; and this written report.

7.2 AI Tool Contributions

Four AI tools were used during the project, each with a different role and a different scope of authority. All AI contributions are disclosed in the spirit of the project's own transparency norm: the same explainability discipline that the system applies to its recommendations is applied here to its construction.

- Claude Opus 4.7 (Anthropic, runtime model) is part of the running system itself: it parses initial user input into structured preferences, maps free-text answers, and writes per-product explanations grounded in the rule-matches list. The trust-boundary discipline (Section 4.1) confines Claude to language work; ranking is never an LLM call. Every Claude call has a deterministic Python fallback so the demo runs without an API key (Section 4.13).
- Claude Code (the Anthropic agent CLI, also running on Opus 4.7) was the build pair-programmer across the 12-week project. It handled scaffolding, refactors, test generation, deterministic fallback implementation, and cross-file consistency passes. All AI-generated code was reviewed line-by-line, refactored, and committed under the author's name. The five-layer architecture, trust-boundary commitment, scenario design, and report structure are the author's intellectual contributions.
- Codex GPT-5.5 (OpenAI) was run as an independent technical validator against the same codebase. It enumerated demo-safety risks, drift between report claims and running code, and places where LLM unavailability would break user-facing behavior. Representative findings and their corresponding fixes are listed in Table 12. Each finding was either fixed in code or dismissed with a written reason; nothing was tacitly accepted. The multi-agent validation method is described in Section 5.5.
- ChatGPT (OpenAI) was used twice during literature review for structured search and for double-checking IEEE citation formatting. No model output was committed verbatim.

Two additional human sources of feedback are also disclosed here for completeness: approximately twelve small-scale user testers (family, friends, and classmates) who walked through the demo end-to-end without prompting; and one industry mentor with 13 years at Amazon spanning Senior Analytics Manager, Senior Business Intelligence Engineer, Data Scientist, and Data Engineer roles, whose input shaped the data discipline and the multi-criteria decision-problem framing (see Section 5.6).

8. References

[1] P. Resnick and H. R. Varian, “Recommender systems,” Communications of the ACM, vol. 40, no. 3, pp. 56–58, 1997.

[2] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” IEEE Transactions on Knowledge and Data Engineering, vol. 17, no. 6, pp. 734–749, 2005.

[3] M. de Gemmis et al., “Semantics-aware content-based recommender systems,” in Recommender Systems Handbook, Springer, 2015, pp. 119–159.

[4] J. Nielsen, “Search: Visible and simple,” Nielsen Norman Group, 2015. [Online]. Available: https://www.nngroup.com/articles/search-visible-simple/

[5] Baymard Institute, “49 Cart abandonment rate statistics,” Baymard Institute, 2024. [Online]. Available: https://baymard.com/lists/cart-abandonment-rate

[6] S. S. Iyengar and M. R. Lepper, “When choice is demotivating: Can one desire too much of a good thing?,” Journal of Personality and Social Psychology, vol. 79, no. 6, pp. 995-1006, 2000.

[7] J. J. Inman and M. Zeelenberg, “Regret in repeat purchase versus switching decisions: The attenuating role of decision justifiability,” Journal of Consumer Research, vol. 29, no. 1, pp. 116-128, 2002.

[8] Amazon, “Amazon Rufus product overview,” Amazon Inc., 2024. [Online]. Available: https://www.aboutamazon.com/news/retail/amazon-rufus

[9] OpenAI, “Buy it in ChatGPT: Instant Checkout and the Agentic Commerce Protocol,” OpenAI, Sept. 29, 2025. [Online]. Available: https://openai.com/index/buy-it-in-chatgpt/

[10] X. Chen et al., “Towards conversational recommendation: A survey,” ACM Computing Surveys, vol. 55, no. 9, pp. 1–37, 2023.

[11] B. Shneiderman, Designing the User Interface: Strategies for Effective Human–Computer Interaction, 6th ed. Boston, MA, USA: Pearson, 2016.

[12] F. Ricci, L. Rokach, and B. Shapira, Eds., Recommender Systems Handbook, 3rd ed. New York, NY, USA: Springer, 2022.

[13] Y. Zhang and X. Chen, “Explainable recommendation: A survey and new perspectives,” Foundations and Trends in Information Retrieval, vol. 14, no. 1, pp. 1–101, 2020.

[14] T. Miller, “Explanation in artificial intelligence: Insights from the social sciences,” Artificial Intelligence, vol. 267, pp. 1–38, 2019.

[15] N. Tintarev and J. Masthoff, “Designing and evaluating explanations for recommender systems,” in Recommender Systems Handbook, F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds. Boston, MA, USA: Springer, 2011, pp. 479-510.

[16] Microsoft Corp., “Playwright for Python documentation,” 2026. [Online]. Available: https://playwright.dev/python/

[17] Anthropic, “Claude documentation and API reference,” Anthropic, 2026. [Online]. Available: https://platform.claude.com/docs/

9. Appendix — Code Listings

Full source code repository: https://github.com/Chenghui-Tan/shopping-assistant. The excerpts in this appendix are reproduced verbatim from the open-source repository above; see each listing's filename header for the canonical path.

9.1 ETL Pipeline Orchestrator (run_pipeline.py — excerpt)

```
async def run(categories: list[str], headed: bool) -> None:
    """Drive scrapers, then normalize and clean. Tolerates per-source
failures."""
    RAW_DIR.mkdir(parents=True, exist_ok=True)
    CLEAN_DIR.mkdir(parents=True, exist_ok=True)
    all_raw: list[list[ScrapedProduct]] = []

    for i, cfg in enumerate(PIPELINE):
        cat = cfg["category"]
        if categories and cat not in categories:
            continue
        products = await scrape_with_fallback(cfg, headed)
        if products:
            products = await boost_with_supplements(products, cfg, headed)
            with open(RAW_DIR / cfg["filename"], "w", encoding="utf-8") as f:
                json.dump(products, f, indent=2, ensure_ascii=False)
            all_raw.append(products)
        if i < len(PIPELINE) - 1:
            await asyncio.sleep(random.uniform(5, 10))   # human-like pause

    if all_raw:
        cleaned = clean_and_combine(all_raw)
        with open(CLEAN_DIR / "products_clean.json", "w", encoding="utf-8") as
f:
            json.dump(cleaned, f, indent=2, ensure_ascii=False)
```

Listing 1: Top-level pipeline orchestration. Iterates the PIPELINE config, scrapes each category with a primary + fallback source, boosts with supplemental queries when counts are low, then defers to clean_and_combine for normalization + deduplication.

9.2 Delivery Normalization (normalize.py — excerpt)

```
def normalize_delivery(raw: str | None) -> int | None:
    """Convert delivery text to number of days from today."""
    if not raw:
        return None
    s = raw.lower().strip()

    if "today" in s or "same day" in s:  return 0
    if "tomorrow" in s:                  return 1

    # "in N days" / "N-day shipping" / "N day"
    for pat in (r"in\s+(\d+)\s+day", r"(\d+)-day", r"(\d+)\s+day"):
        m = re.search(pat, s)
        if m:
            return int(m.group(1))
```

```
    # Absolute date: "by Mon, Mar 14" / "arrives Mar 14"
    m = re.search(
        r"(?:by|arrives?|get it by)[^a-z]*"
        r"([a-z]{3})\w*\s*[,.]?\s*(\d{1,2})",
        s,
    )
    if m:
        return _days_until_month_day(m.group(1), m.group(2))
    return None
```

Listing 2: Delivery-string parser. Returns None on unparseable text; the ranker treats None as a neutral 0.5 sub-score so missing data does not bias ranking. Real implementation handles four more formats (date ranges, standalone month-day, etc.).

9.3 Constraint-Aware Ranking (recommendation_engine_refactored.py — excerpt)

```
def recommend_products(prefs: dict, top_n: int = 10) -> list[dict]:
    """Filter → rank → top_n; relax constraints if too few survive."""
    products = load_products()

    result = _try_ranked(products, prefs, top_n)
    if result is not None:                      # full constraints
        return result

    # Fallback 1: drop delivery
    relaxed = {k: v for k, v in prefs.items() if k != "delivery_days_max"}
    result = _try_ranked(products, relaxed, top_n)
    if result is not None:
        return result

    # Fallback 2: drop price + delivery (category only)
    relaxed = {k: v for k, v in prefs.items()
               if k not in ("delivery_days_max", "price_max")}
    result = _try_ranked(products, relaxed, top_n)
    if result is not None:
        return result

    # Fallback 3: ignore category — top by shared score
    base = {"priority": prefs.get("priority", "balanced")}
    return rank_products(load_products(), base)[:top_n]
```

Listing 3: Three-tier fallback. Each tier relaxes one constraint; the ranker never returns an empty list as long as the dataset is non-empty.

9.4 Scoring Rule (water_bottle gym example)

```
def _score_water_bottle(product, preferences, features):
    use_case = preferences.get("use_case", "")
    score, matches = 0.0, []

    if use_case == "gym":
        gym_match = (
            features["lightweight"]
            or _title_has(product, "freesip", "owala", "stainless steel")
```

```
        )
        score = _apply_rule(score, matches, gym_match, "gym_suitable")
        score = _apply_rule(score, matches, features["insulated"],
"insulated_gym")

    if preferences.get("insulated"):
        score = _apply_rule(score, matches, features["insulated"],
                            "insulated", penalty=_SOFT_PENALTY)
    # ... size_preference, material_preference, drinking_style,
leak_proof_preferred ...
    return score, matches
```

Listing 4: Category-specific scoring for the water-bottle gym use case. Each matched rule adds +0.10; each user-requested-but-missing feature subtracts the soft penalty −0.05. Real function has parallel branches for daily / outdoor / kids and for the four explicit-feature preferences.

9.5 FastAPI Routes (main.py — excerpt)

```
@app.post("/session/start")
def start_session(body: StartBody):
    # Best-effort LLM parse; fall back to chip selection on any failure.
    try:
        parsed = claude_client.parse_initial_input(body.text)
        category = parsed.get("category", "unknown")
        preferences = parsed.get("preferences", {})
    except Exception:
        category, preferences = "unknown", {}

    # Deterministic category fallback when the LLM didn't classify.
    if category in ("unknown", "", None):
        inferred = _classify_category(body.text)
        if inferred:
            category = inferred

    # Implicit preference inference from the raw text.
    raw_lower = (body.text or "").lower()
    if any(k in raw_lower for k in ("leak", "spill")):
        preferences["leak_proof_preferred"] = True
    if any(k in raw_lower for k in ("heavy", "bulky", "easy to carry",
"portable")):
        preferences.setdefault("size_preference", "lightweight")

    s = session_store.create_session(body.text, category, preferences)
    return {
        "session_id": s["session_id"],
        "category": category or None,
        "next_question": _next_question(s) if category else None,
        "chips": None if category else CATEGORY_CHIPS,
        "preferences": preferences,
        "reply": _empathic_reply(category, preferences, body.text),
    }
```

Listing 5: Session-start endpoint. The LLM is best-effort; on failure a regex classifier (_classify_category) routes the input. Implicit preferences are inferred from the raw text so the chip-pre-fill in Stage 2 can reflect what the user already said.

9.6 Trust Boundary in the Recommendation Pipeline

```
def _run_recommendations(session: dict) -> tuple[list[dict], str]:
    prefs = {**session["preferences"], "category": session["category"]}
    # Deterministic — never an LLM call. Returns (products, relaxation_tier).
    products, relaxation = engine.recommend_with_relaxation(prefs, top_n=12)
    try:
        explanations = claude_client.write_explanations(
            products, session["preferences"], session["raw_input"])
    except Exception:
        explanations = []                                          # LLM
unavailable
    explanations = _normalize_explanations(products, explanations) # exactly
one per product

    formatted = []
    for i, p in enumerate(products):
        row = _format_product(p, explanations[i])
        row["pref_checks"] = _preference_checks(p, prefs)          # ✓ / ✗ / ?
per chip
        formatted.append(row)
    session_store.set_recommendations(session["session_id"], formatted)
    return formatted, relaxation
```

Listing 6: Where the trust boundary lives in code. Ranking is a pure function call; explanation generation is best-effort and gracefully falls back to the deterministic explainer.

9.7 Curated 3-Pick Differentiation (curated_picks excerpt)

```
def curated_picks(ranked, category):
    """Return Best Fit, Budget Pick, Stretch — with guard rails."""
    # Score lookup tolerant of both shapes: raw engine output uses `_score`,
    # the FastAPI route hands us already-formatted products where the same
    # number is stored as `score`. Without this fallback the 75% floor would
    # silently collapse to 0 when called from the route.
    def _sc(p):
        s = p.get("_score")
        return (s if s is not None else p.get("score")) or 0.0

    best = ranked[0]
    picks = [(best, "Best Fit",
              "Highest match score across all your stated preferences.")]

    # Budget Pick: cheapest comparable product that still scores >= 75% of
best.
    score_floor = _sc(best) * 0.75
    budget_pool = sorted(
        [p for p in ranked[1:] if _sc(p) >= score_floor],
        key=lambda p: p.get("price") or 1e9)
    if budget_pool and budget_pool[0]["price"] + 5 <= best["price"]:
        saving = best["price"] - budget_pool[0]["price"]
        picks.append((budget_pool[0], "Budget Pick",
                      f"${budget_pool[0]['price']:.0f} — saves ${saving:.0f} vs
Best Fit."))
```

```
    # Stretch Pick: category-specific, requires a meaningful margin over Best
Fit.
    if category == "smart_display":
        with_screen = [p for p in ranked if p.get("screen_inches")]
        if with_screen:
            cand = max(with_screen, key=lambda p: p["screen_inches"])
            if cand["screen_inches"] >= (best.get("screen_inches") or 0) + 2:
                picks.append((cand, "Large Screen Pick",
                              f'{cand["screen_inches"]}" — easier from across
the kitchen.'))
    # ... water_bottle / kitchen_organizer branches similar ...
    return [{**p, "pick_label": l, "pick_reason": r} for p, l, r in picks]
```

Listing 7: Differentiated three-pick selection. The score_floor and margin tests prevent a $5 sensor from posing as a Budget Pick and prevent a minor-difference product from claiming the Stretch slot. The local _sc helper was added during the post-audit pass (§5.7) — the route hands curated_picks products with `score` (renamed from `_score` by _format_product), and without the dual-key read the 75% floor silently collapsed to 0.

9.8 Preference Match Checklist (per-product evaluator)

```
def _preference_checks(p, prefs):
    """Return [{label, status: match|miss|unknown}] for every user-picked
chip."""
    rows = []
    cat = p.get("category")
    feats = p.get("_inferred_features") or {}
    rules = set(p.get("_category_rule_matches") or [])

    # Use case fit: did the cat-specific rule for this use case actually fire?
    use_case = prefs.get("use_case")
    if use_case:
        wb_rule = {"gym": "gym_suitable", "outdoor": "outdoor_capacity",
                   "kids": "kids_design", "daily": "daily_use"}
        cases = use_case if isinstance(use_case, list) else [use_case]
        matched = any([
            cat == "water_bottle"  and wb_rule.get(u) in rules,
            cat == "smart_display" and u == "cooking"
                                   and "kitchen_hub_or_recipe" in rules,
            cat == "smart_display" and u == "family"
                                   and "family_scheduling" in rules,
            cat == "smart_display" and u == "entertainment"
                                   and "entertainment_features" in rules,
        ] for u in cases)
        rows.append(_check("Use case fit",
                           "match" if matched else "unknown",
                           ", ".join(cases)))

    # ... price_max, material, drinking_style, insulated, size, ecosystem,
    #     screen_size, no_camera, organizer_material, visibility — each a
row ...
    return rows
```

Listing 8: Per-pick preference checklist. Status comes from the same rule_matches the ranker used, so a row can only say 'match' if the rule fired.

9.9 Deterministic Refine Parser (regex fallback)

```
_SUPP_RULES_GLOBAL = [
    (r"(?:under|less than|below|cheaper than)\s*\$?\s*(\d+)", "price_max",
"_int"),
    (r"\bno\s+camera\b|\bwithout\s+camera\b",                "privacy_camera",
"no_camera"),
    (r"\b(?:works\s+with\s+)?google\b|\bnest\s+hub\b",       "voice_ecosystem",
"google"),
    (r"\bleak[-\s]?proof\b|\bno\s+(?:spill|leak)\b",
"leak_proof_preferred", True),
    # ... 25 more patterns ...
]

def _deterministic_parse_supplement(text, category):
    """Pattern-match a refine string into the same shape Claude would emit."""
    lower = (text or "").lower()
    updates = {}
    for pat, key, val in _SUPP_RULES_GLOBAL + _SUPP_RULES_BY_CAT.get(category,
[]):
        if key in updates:
            continue
        m = re.search(pat, lower)
        if not m:
            continue
        updates[key] = int(m.group(1)) if val == "_int" else val
    return {"preference_updates": updates,
            "ai_response": "Got it — updating " + ", ".join(updates) + "."}
```

Listing 9: Regex fallback parser. Three categories of phrase are handled — global (price, delivery, privacy, ecosystem), and per-category (size, material, drinking style, structure, visibility). Each Claude call has a parallel fallback like this; the demo runs end-to-end with ANTHROPIC_API_KEY=placeholder.